

Chatbot: Security

Objective: Improve the current chat system by enhancing security measures.

INTRODUCTION

This exercise focuses on introducing additional security measures to the updated chat system with authentication. As a recap, the chatbot system is a client-server system that allows multiple clients to connect to a single server via Python's socket package. The server handles multiple client connections as well as the message flow. Each client is authenticated by the server utilizing a username and password combination. The server may receive a message from multiple clients at one time but will handle the messages in the order that they are received.

SECURITY IMPLEMENTATION

We have augmented our application utilizing 3 techniques. Firstly, we have implemented SSL/TLS (Secure Socket Layer / Transport Security Layer) to secure our socket connections. Secondly, we have implemented hashing of our passwords. Thirdly, we have utilized "salt" to complement and improve the password protection.

SSL/TLS has been implemented utilizing the python "ssl" package. In simple terms, we use this protocol to wrap socket communication. It is a certificate based and key based protocol. Both the client and the server need to secure their ends for SSL/TLS to function properly. There are other features within the SSL protocol that are not completely implemented and are set to "Developer Mode" but would be configured if this application went live.

Hashing is a technique to encrypt passwords to make them much more difficult to use and recover by malicious parties. We utilize python's "hashlib" package. Our application is utilizing PBKDF2 (Password-Based Key Derivation Function 2) with SHA-256 algorithm for hashing.

Salt is a technique that strengthens hashing and provides more protection against attacks. Salt is a random sequence of bytes that gets added to a new password. So, when a user generates a password, their password gets a new generated "salt" that gets stored with the hash. We utilized the "os" package, specifically its urandom(32) method, to generate our "salt" contain 32 bytes of random data.

Both the Salt and the Hash are stored in a dictionary as a pair utilizing hex coding. Note, that we never store the original password (please note the user_manager.py does, in fact, have the passwords of our test accounts. In a real-world application the user would be prompted to make their own). When a user tries to authenticate, a new hash is generated and is used in conjunction with the salt. These are compared to the stored value. If they match, then we proceed with authentication.

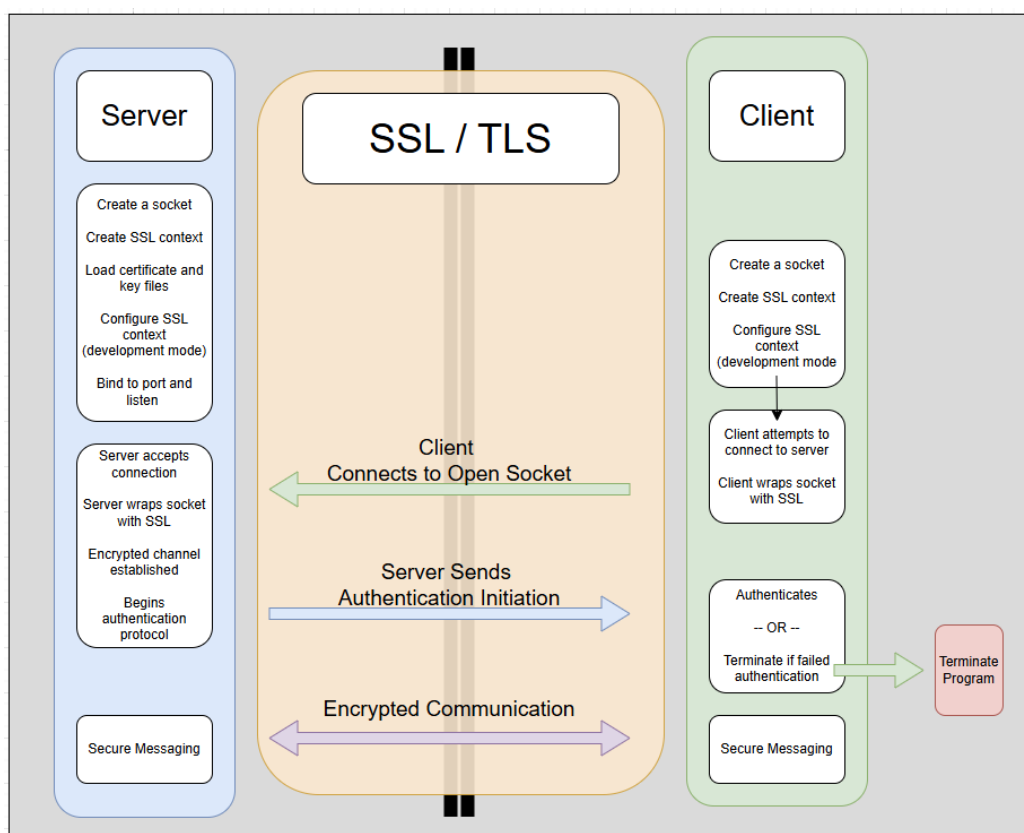


Figure 1: SSL/TLS flow in the chat system.

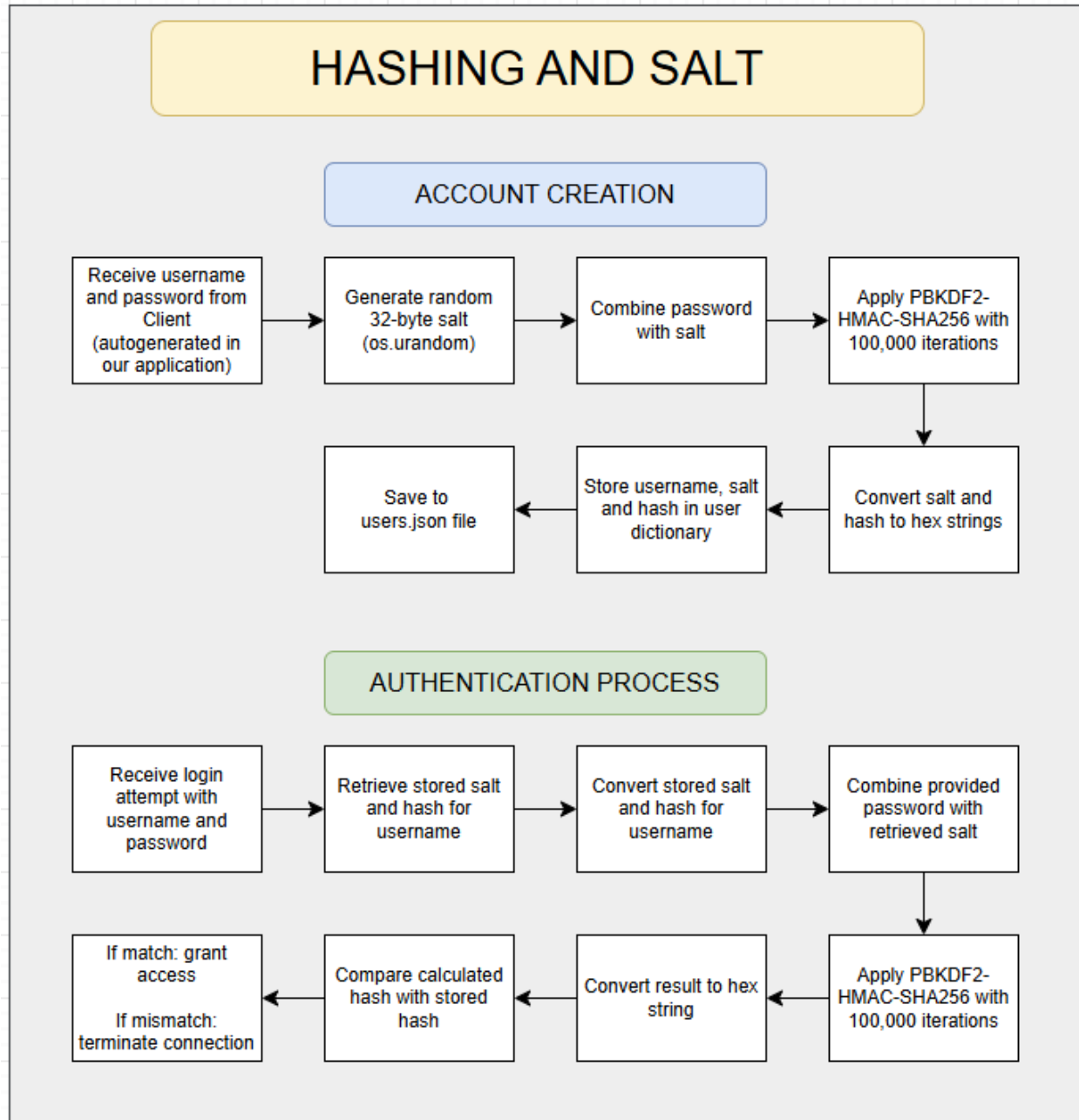


Figure 2: Salt/Hashing flow in the chat system.

USER GUIDE

1) Run “*python server.py*”. The server begins and listens for client connections. In this case it is listening on port 5555.

```
PS C:\Users\dleon\Documents\SCSU_SY25 SP\CSCI 312 Fundamentals of Distributed Systems\Labs\03 Chat System\chat_system_v3\csci312_chat_system> python server.py
2025-03-27 13:03:42,710 - INFO - Secure server is listening on port 5555...
```

2) [Server – Left / Client – Right] Run “*python client.py*” in a separate terminal. A client attempts to login to the server with random credentials but fails authentication. The server is notified. The client application terminates.

```
PS C:\Users\dleon\Documents\SCSU_SY25 SP\CSCI 312 Fundamentals of Distributed Systems\Labs\03 Chat System\chat_system_v3\csci312_chat_system> python server.py
2025-03-27 13:03:42,710 - INFO - Secure server is listening on port 5555...
Secure connection from ('127.0.0.1', 52167) has been established.
Authenticating asdfdsa
User asdfdsa not found
Authentication failed for ('127.0.0.1', 52167). Connection closed.

PS C:\Users\dleon\Documents\SCSU_SY25 SP\CSCI 312 Fundamentals of Distributed Systems\Labs\03 Chat System\chat_system_v3\csci312_chat_system> python client.py
Username: asdfdsa
Password: asdffsd
Authentication failed
```

3) [Server – Left / Client – Right] Run the program again as a client. The client attempts to login to the server. This is a known client. The client “test” authenticates and begins chat.

```
PS C:\Users\dleon\Documents\SCSU_SY25 SP\CSCI 312 Fundamentals of Distributed Systems\Labs\03 Chat System\chat_system_v3\csci312_chat_system> python server.py
2025-03-27 13:03:42,710 - INFO - Secure server is listening on port 5555...
Secure connection from ('127.0.0.1', 52167) has been established.
Authenticating asdfdsa
User asdfdsa not found
Authentication failed for ('127.0.0.1', 52167). Connection closed.
Secure connection from ('127.0.0.1', 52172) has been established.
Authenticating test
Authentication successful for user: test
[]

PS C:\Users\dleon\Documents\SCSU_SY25 SP\CSCI 312 Fundamentals of Distributed Systems\Labs\03 Chat System\chat_system_v3\csci312_chat_system> python client.py
Username: asdfdsa
Password: asdffsd
Authentication failed

PS C:\Users\dleon\Documents\SCSU_SY25 SP\CSCI 312 Fundamentals of Distributed Systems\Labs\03 Chat System\chat_system_v3\csci312_chat_system> python client.py
Username: test
Password: test
Authentication successful

Enter message: []
```

4) [Client – Left / Server - Right] In a new terminal, run another client. The new client attempts to login to the server. This is also a known client. The client “user1” authenticates and begins chat. Clients “test” and “user1” are connected to the server simultaneously.

```
PS C:\Users\dleon\Documents\SCSU_SY25 SP\CSCI 312 Fundamentals of Distributed Systems\Labs\03 Chat System\chat_system_v3\csci312_chat_system> python client.py
Username: user1
Password: password1
Authentication successful

Enter message: []

PS C:\Users\dleon\Documents\SCSU_SY25 SP\CSCI 312 Fundamentals of Distributed Systems\Labs\03 Chat System\chat_system_v3\csci312_chat_system> python server.py
2025-03-27 13:03:42,710 - INFO - Secure server is listening on port 5555...
Secure connection from ('127.0.0.1', 52167) has been established.
Authenticating asdfdsa
User asdfdsa not found
Authentication failed for ('127.0.0.1', 52167). Connection closed.
Secure connection from ('127.0.0.1', 52172) has been established.
Authenticating test
Authentication successful for user: test
Secure connection from ('127.0.0.1', 52186) has been established.
Authenticating user1
Authentication successful for user: user1
[]
```

5) [Server – Left / “test” – Right] The client “test” messages the server with “Hello” and hits the Enter/Return key. This is received and displayed to the server.

```
PS C:\Users\dleon\Documents\SCSU_SY25 SP\CSCI 312 Fundamentals of Distributed Systems\Labs\03 Chat System\chat_system_v3\csci312_chat_system> python server.py
2025-03-27 13:03:42,710 - INFO - Secure server is listening on port 5555...
Secure connection from ('127.0.0.1', 52167) has been established.
Authenticating asdfdsa
User asdfdsa not found
Authentication failed for ('127.0.0.1', 52167). Connection closed.
Secure connection from ('127.0.0.1', 52172) has been established.
Authenticating test
Authentication successful for user: test
Secure connection from ('127.0.0.1', 52186) has been established.
Authenticating user1
Authentication successful for user: user1
Client ('127.0.0.1', 52172): Hello
Enter response: █

PS C:\Users\dleon\Documents\SCSU_SY25 SP\CSCI 312 Fundamentals of Distributed Systems\Labs\03 Chat System\chat_system_v3\csci312_chat_system> python client.py
Username: asdfdsa
Password: asdfdsa
Authentication failed

PS C:\Users\dleon\Documents\SCSU_SY25 SP\CSCI 312 Fundamentals of Distributed Systems\Labs\03 Chat System\chat_system_v3\csci312_chat_system> python client.py
Username: test
Password: test
Authentication successful

Enter message: Hello
█
```

6) [“user1” – Left / Server – Right] The client “user1” messages the server with “Wait! It’s my turn!!!”. This is received and displayed to the server.

```
PS C:\Users\dleon\Documents\SCSU_SY25 SP\CSCI 312 Fundamentals of Distributed Systems\Labs\03 Chat System\chat_system_v3\csci312_chat_system> python client.py
Username: user1
Password: password1
Authentication successful

Enter message: Wait! It's my turn!!!
█

PS C:\Users\dleon\Documents\SCSU_SY25 SP\CSCI 312 Fundamentals of Distributed Systems\Labs\03 Chat System\chat_system_v3\csci312_chat_system> python server.py
2025-03-27 13:03:42,710 - INFO - Secure server is listening on port 5555...
Secure connection from ('127.0.0.1', 52167) has been established.
Authenticating asdfdsa
User asdfdsa not found
Authentication failed for ('127.0.0.1', 52167). Connection closed.
Secure connection from ('127.0.0.1', 52172) has been established.
Authenticating test
Authentication successful for user: test
Secure connection from ('127.0.0.1', 52186) has been established.
Authenticating user1
Authentication successful for user: user1
Client ('127.0.0.1', 52172): Hello
Client ('127.0.0.1', 52186): Wait! It's my turn!!!
█
```

7) [Server – Left / “test” – Right] The server sends a message of “Hi. The server’s message responds to client “test” message “Hello” and not client “user1” message “Wait! It’s my turn!!!”.

```

Distributed Systems\Labs\03 Chat System\chat_system_v3\csci312_chat
_system> python server.py
2025-03-27 13:03:42,710 - INFO - Secure server is listening on port
5555...
Secure connection from ('127.0.0.1', 52167) has been established.
Authenticating asdfdsa
User asdfdsa not found
Authentication failed for ('127.0.0.1', 52167). Connection closed.
Secure connection from ('127.0.0.1', 52172) has been established.
Authenticating test
Authentication successful for user: test
Secure connection from ('127.0.0.1', 52186) has been established.
Authenticating user1
Authentication successful for user: user1
Client ('127.0.0.1', 52172): Hello
Client ('127.0.0.1', 52186): Wait! It's my turn!!!
Hi
Enter response:

PS C:\Users\dleon\Documents\SCSU\SY25 SP\CSCI 312 Fundamentals of
Distributed Systems\Labs\03 Chat System\chat_system_v3\csci312_chat
_system> python client.py
Username: asdfdsa
Password: asdfdsa
Authentication failed

PS C:\Users\dleon\Documents\SCSU\SY25 SP\CSCI 312 Fundamentals of
Distributed Systems\Labs\03 Chat System\chat_system_v3\csci312_chat
_system> python client.py
Username: test
Password: test
Authentication successful

Enter message: Hello
Server: Hi
Enter message:

```

8) [“user1” – Left / Server – Right] The server sends a message of “I’m sorry. Please be patient” which is received by “user1” matching order.

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  PORTS  TERMINAL
PS C:\Users\dleon\Documents\SCSU\SY25 SP\CSCI 312 Fundamentals of
Distributed Systems\Labs\03 Chat System\chat_system_v3\csci312_chat
_system> python client.py
Username: user1
Password: password1
Authentication successful

Enter message: Wait! It's my turn!!!
Server: I'm sorry. Please be patient.
Enter message:

PS C:\Users\dleon\Documents\SCSU\SY25 SP\CSCI 312 Fundamentals of
Distributed Systems\Labs\03 Chat System\chat_system_v3\csci312_chat
_system> python server.py
2025-03-27 13:03:42,710 - INFO - Secure server is listening on port
5555...
Secure connection from ('127.0.0.1', 52167) has been established.
Authenticating asdfdsa
User asdfdsa not found
Authentication failed for ('127.0.0.1', 52167). Connection closed.
Secure connection from ('127.0.0.1', 52172) has been established.
Authenticating test
Authentication successful for user: test
Secure connection from ('127.0.0.1', 52186) has been established.
Authenticating user1
Authentication successful for user: user1
Client ('127.0.0.1', 52172): Hello
Client ('127.0.0.1', 52186): Wait! It's my turn!!!
Hi
Enter response: I'm sorry. Please be patient.

```

TESTING

Manual testing was conducted. The Usernames and Passwords were predetermined and hardcoded.

Test Set 1: Unknown Username / Unknown Password

- asdasd / asdasd
- bob / lkjlkj
- user3 / lalala

Results: PASSED – failed to authenticate.

Test Set 2: Unknown Username / Known Password

- bob / test
- bob / password1
- bob / password2

Results: PASSED – failed to authenticate.

Test Set 3: Known Username / Unknown Password

- test / asdasd
- test / lkjlkj
- test / password3

Results: PASSED – failed to authenticate.

Test Set 4: Known Username / Known Password

- test / test
- user1 / password1
- user2 / password2

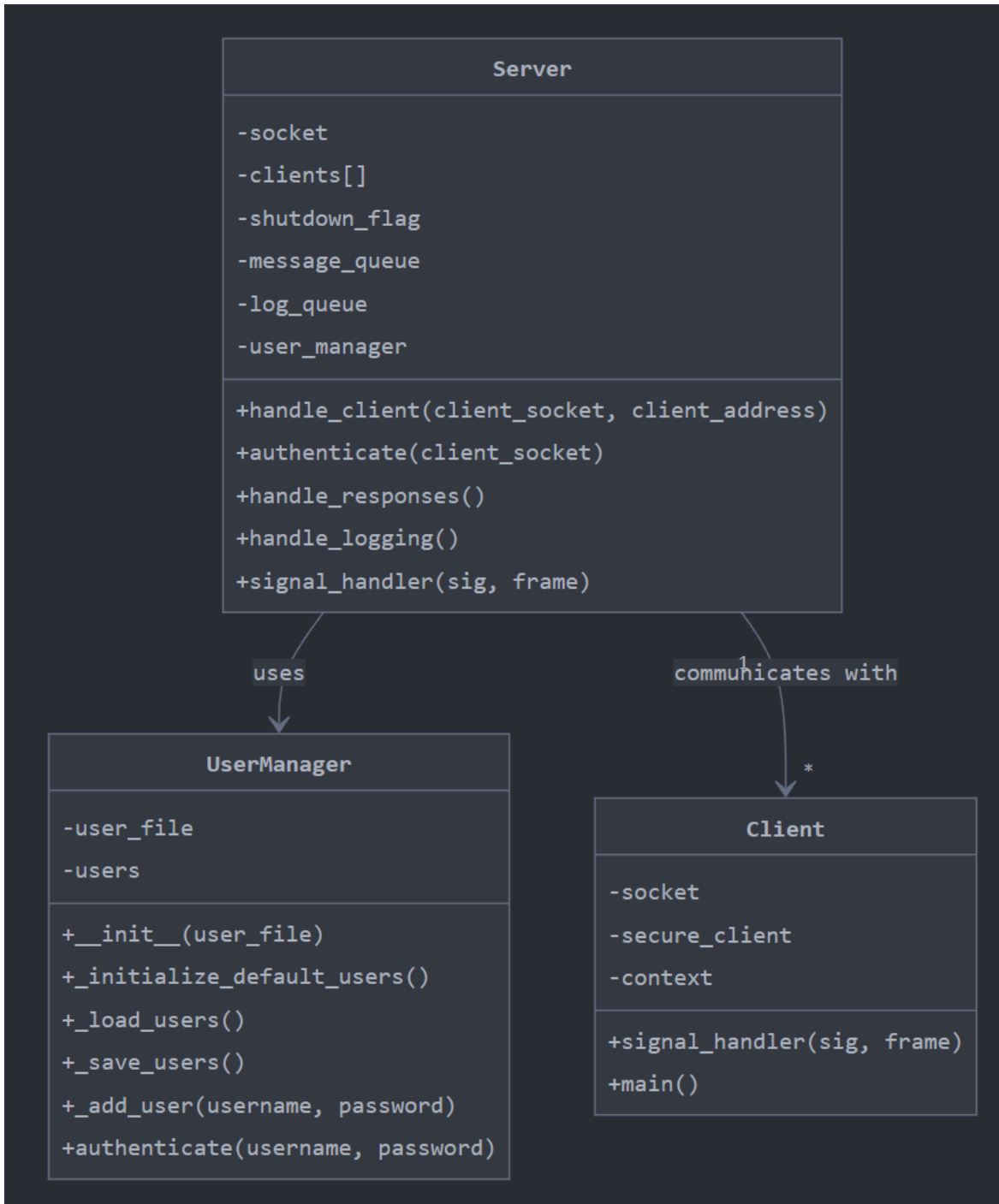
Results: PASSED – properly authenticated.

CLOSING

Our newly added security measures improve upon our previous systems. SSL/TLS provide additional security for our socket connections and message transmission. The addition of salt and hashing has greatly improved user privacy by eliminating the need to directly store the user password, but more than that, stores an encrypted version with additional randomness to make it extremely difficult if not impossible to acquire.

There are additional improvements that can be made to the system as a whole. In terms of security, there are some relatively simple additions and adjustments that can further secure the system. Firstly, we can enforce a password policy to ensure passwords are appropriately secure, such as requiring a mix of letters, numbers, symbols, etc. Secondly, we could implement an MFA or two-factor authentication through security questions or PINs. Thirdly, we can implement a system to rate login attempts and create an account locking mechanism. Fourthly, we could add session time outs for inactivity. Lastly, we can re-evaluate the overall application architecture to ensure isolation and separation of critical information and processes. There are many more ways to further secure the system, but these are relatively simple to implement and integrate.

UML DIAGRAM



CODE BASE

client.py

```
import socket
import signal
import sys
import ssl

# Client setup
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Set up SSL context
context = ssl.create_default_context()
# For development with self-signed certificates, disable
certificate verification
context.check_hostname = False
context.verify_mode = ssl.CERT_NONE

# Connect with SSL
try:
    client.connect(('localhost', 5555))
    secure_client = context.wrap_socket(client,
server_hostname='localhost')
except Exception as e:
    print(f"Connection error: {e}")
    sys.exit(1)

def signal_handler(sig, frame):
    print("\nShutting down secure client...")
    try:
        secure_client.send("logout".encode('utf-8'))
        secure_client.close()
    except:
        pass
    sys.exit(0)

signal.signal(signal.SIGINT, signal_handler)

# Authentication
try:
    response = secure_client.recv(1024).decode('utf-8')
    print(response, end='') # Print "Username: "
    username = input()
    secure_client.send(username.encode('utf-8'))

    response = secure_client.recv(1024).decode('utf-8')
```

```

print(response, end='') # Print "Password: "
password = input()
secure_client.send(password.encode('utf-8'))

response = secure_client.recv(1024).decode('utf-8')
print(response)
if "Authentication failed" in response:
    secure_client.close()
    sys.exit(0)
except ssl.SSLError as e:
    print(f"SSL Error during authentication: {e}")
    secure_client.close()
    sys.exit(1)
except Exception as e:
    print(f"Error during authentication: {e}")
    secure_client.close()
    sys.exit(1)

# Communication loop
while True:
    try:
        message = input("Enter message: ")
        secure_client.send(message.encode('utf-8'))
        if message.lower() == "logout":
            break
        response = secure_client.recv(1024).decode('utf-8')
        if not response:
            break
        print(f"Server: {response}")
    except ssl.SSLError as e:
        print(f"SSL Error: {e}")
        break
    except ConnectionResetError:
        print("Connection was reset by the server.")
        break
    except Exception as e:
        print(f"Error: {e}")
        break

secure_client.close()
print("Connection closed.")

```

server.py

```
import socket
import threading
import signal
import sys, time
import logging
import queue
import ssl
from user_manager import UserManager

# Configure logging
logging.basicConfig(level=logging.INFO, format='%(asctime)s -
%(levelname)s - %(message)s')

# Server setup
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(('localhost', 5555))
server.listen(5)
server.settimeout(1) # Set a timeout for the accept call

# Set up SSL context
context = ssl.create_default_context(ssl.Purpose.CLIENT_AUTH)
context.load_cert_chain(certfile="server.cert",
keyfile="server.key")
context.check_hostname = False #Currently false for development
context.verify_mode = ssl.CERT_NONE

logging.info("Secure server is listening on port 5555...")

clients = []
shutdown_flag = False
message_queue = queue.Queue()
log_queue = queue.Queue()
user_manager = UserManager()

def authenticate(client_socket):
    client_socket.send("Username: ".encode('utf-8'))
    username = client_socket.recv(1024).decode('utf-8')
    client_socket.send("Password: ".encode('utf-8'))
    password = client_socket.recv(1024).decode('utf-8')

    if user_manager.authenticate(username, password):
        client_socket.send("Authentication
successful\n".encode('utf-8'))
        log_queue.put(f"Authentication successful for user:
{username}")
```

```

        return True
    else:
        client_socket.send("Authentication
failed\n".encode('utf-8'))
        # Remove from clients list before closing
        if client_socket in clients:
            clients.remove(client_socket)
        client_socket.close()
        return False

def handle_client(client_socket, client_address):
    log_queue.put(f"Secure connection from {client_address} has
been established.")
    if not authenticate(client_socket):
        log_queue.put(f"Authentication failed for
{client_address}. Connection closed.")
        return

    while True:
        try:
            message = client_socket.recv(1024).decode('utf-8')
            if not message:
                break
            if message.lower() == "logoff":
                break
            log_queue.put(f"Client {client_address}: {message}")
            message_queue.put((client_socket, message))
        except (ConnectionResetError, ssl.SSLError):
            break
        client_socket.close()
        clients.remove(client_socket)
        log_queue.put(f"Connection from {client_address} has been
closed.")

def handle_responses():
    while not shutdown_flag:
        try:
            if not message_queue.empty():
                print("Enter response: ", end='', flush=True)
                client_socket, message =
message_queue.get(timeout=1)
                response = input()
                client_socket.send(response.encode('utf-8'))
            else:
                # No messages to respond to, short sleep to
prevent CPU spinning
                time.sleep(0.1)

```

```

        except queue.Empty:
            continue

def handle_logging():
    while not shutdown_flag:
        try:
            log_message = log_queue.get(timeout=1)
            print(f"\r{log_message}", flush=True)
        except queue.Empty:
            continue

def signal_handler(sig, frame):
    global shutdown_flag
    log_queue.put("Shutting down secure server...")
    shutdown_flag = True
    for client in clients[:]: # Create a copy of the list to
        # iterate over
        try:
            client.send("Server is shutting
down...".encode('utf-8'))
        except:
            # Skip errors for closed sockets
            pass
        finally:
            try:
                client.close()
            except:
                pass
    server.close()
    sys.exit(0)

signal.signal(signal.SIGINT, signal_handler)

response_thread = threading.Thread(target=handle_responses)
response_thread.start()

logging_thread = threading.Thread(target=handle_logging)
logging_thread.start()

while not shutdown_flag:
    try:
        # Accept connection
        client_socket, client_address = server.accept()
        # Wrap socket with SSL
        secure_socket = context.wrap_socket(client_socket,
server_side=True)
        clients.append(secure_socket)

```

```
        client_thread = threading.Thread(target=handle_client,
args=(secure_socket, client_address))
        client_thread.start()
    except socket.timeout:
        continue
    except ssl.SSLError as e:
        log_queue.put(f"SSL Error: {e}")
    except Exception as e:
        log_queue.put(f"Error: {e}")
        break

server.close()
```

user_manager.py

```
import hashlib
import os
import json

class UserManager:
    def __init__(self, user_file='users.json'):
        self.user_file = user_file
        self.users = {}
        self._load_users()

        # Initialize with default users if file doesn't exist
        yet
        if not self.users:
            self._initialize_default_users()

    def _initialize_default_users(self):
        """Initialize with default users from the original
server implementation"""
        default_users = {
            "test": "test",
            "user1": "password1",
            "user2": "password2"
        }

        for username, password in default_users.items():
            self._add_user(username, password)

        self._save_users()

    def _load_users(self):
        """Load users from file if it exists, otherwise start
with empty dict"""
        try:
            with open(self.user_file, 'r') as f:
                self.users = json.load(f)
        except (FileNotFoundError, json.JSONDecodeError):
            self.users = {}

    def _save_users(self):
        """Save users to file"""
        with open(self.user_file, 'w') as f:
            json.dump(self.users, f)

    def _add_user(self, username, password):
        """Internal method to add a user with hashed password"""
```

```

        salt = os.urandom(32)

        # Create hash with salt
        key = hashlib.pbkdf2_hmac(
            'sha256',
            password.encode('utf-8'),
            salt,
            100000
        )

        # Store the user with salt and hash
        self.users[username] = {
            "salt": salt.hex(),
            "hash": key.hex()
        }

def authenticate(self, username, password):
    print(f"Authenticating {username}")
    if username not in self.users:
        print(f"User {username} not found")
        return False

    # Get stored salt and hash
    stored = self.users[username]
    #print(f"Stored data: {stored}")

    salt = bytes.fromhex(stored["salt"])
    stored_hash = stored["hash"]

    # Hash the provided password with the same salt
    key = hashlib.pbkdf2_hmac(
        'sha256',
        password.encode('utf-8'),
        salt,
        100000
    )

    calculated_hash = key.hex()
    #print(f"Calculated hash: {calculated_hash}")
    #print(f"Stored hash: {stored_hash}")

    # Compare hashes
    return calculated_hash == stored_hash

```